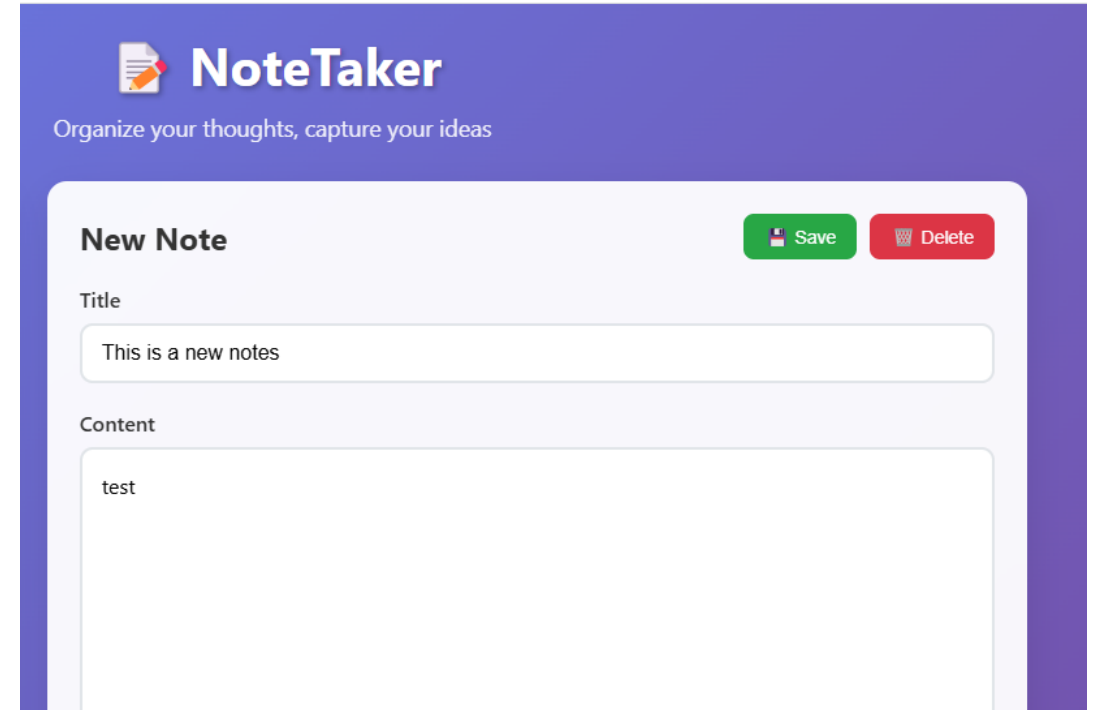
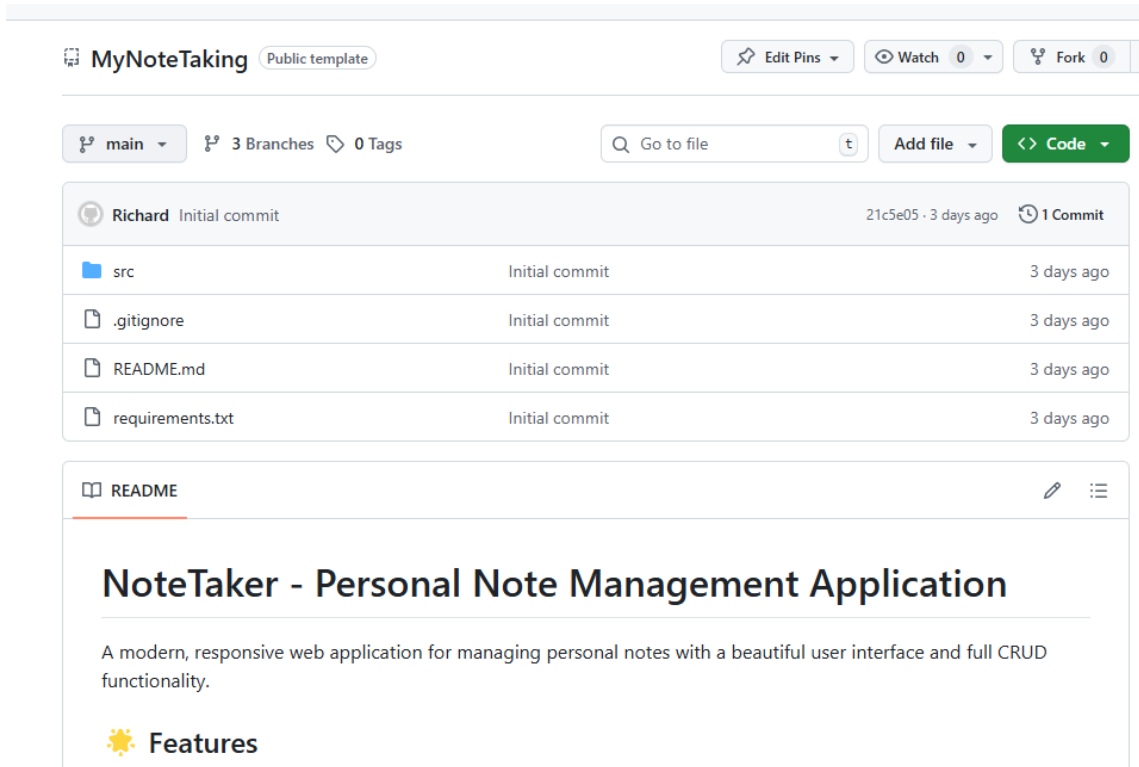


Exercise: AI Assisted Coding

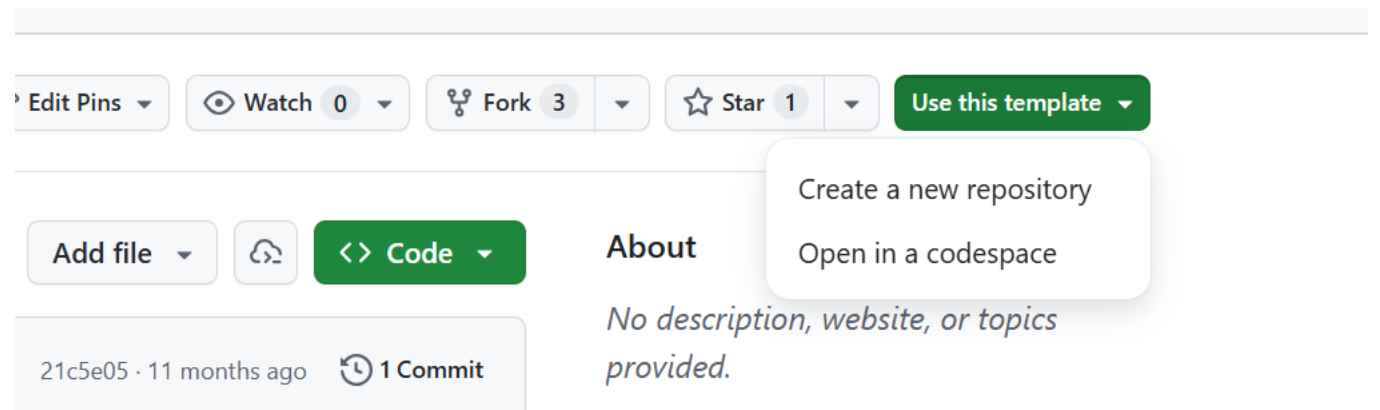
Overview



Github Repository: <https://github.com/HKPolyUSE/MyNoteTaking>

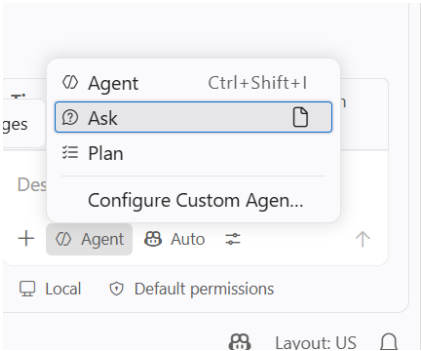
The note taking app developed by Manus AI: <https://3dhkiloc88dkk.manus.space/>

- Approach 1:
 - Use git to clone this repository to your local computer and work locally
- Approach 2:
 - Use this template -> Create a new repository. Create codespaces from the repository
- You may use Github Copilot, Opencode, other coding agents to complete this exercise

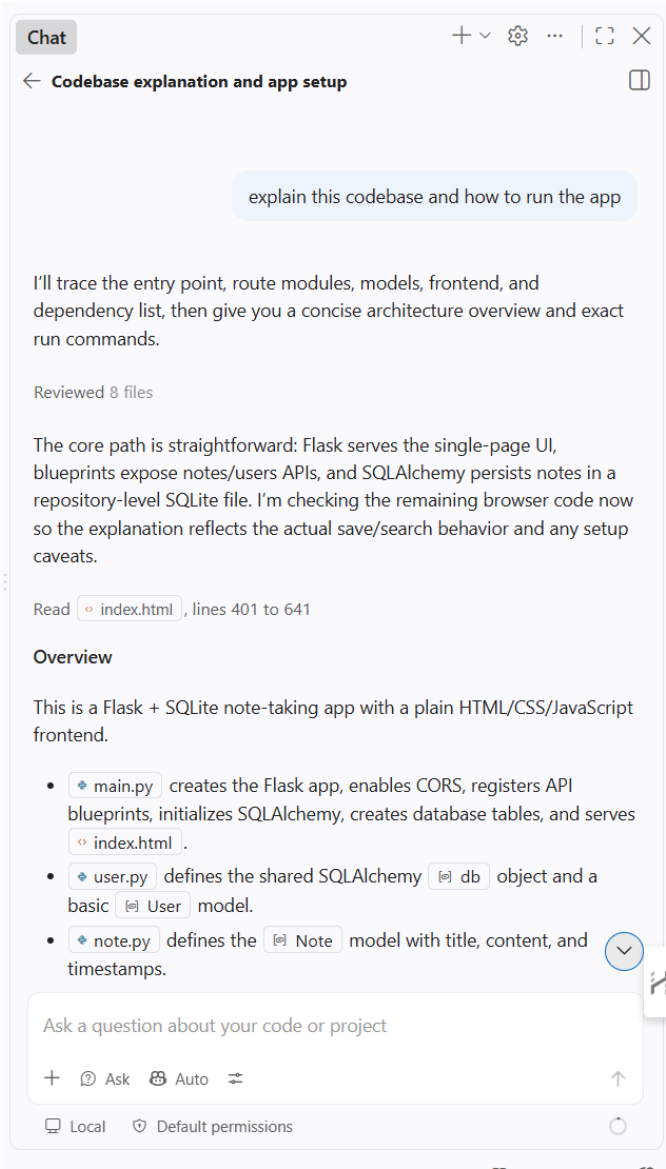


Explore the codebase

- Use **Ask** mode in GitHub Copilot
- Follow the suggested steps to run the app

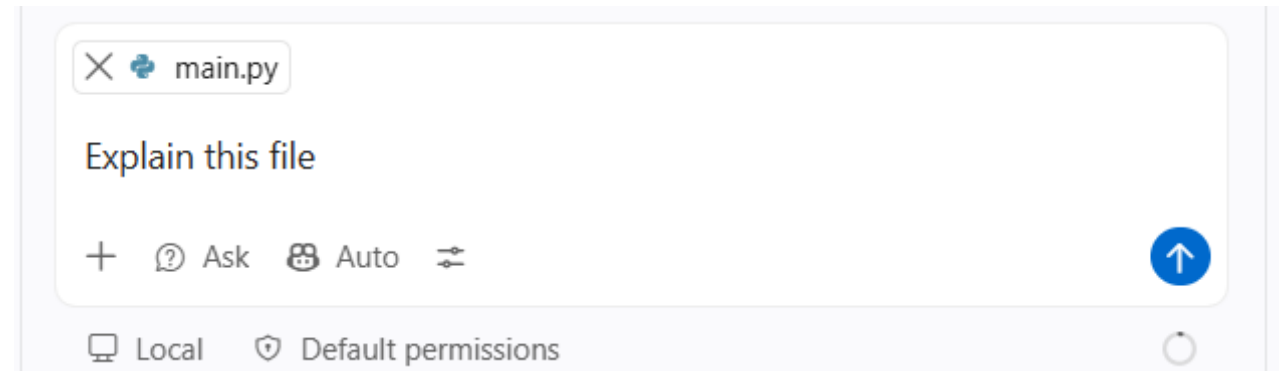


Ask	Conversational Assistance The standard chat interface where you ask questions, request explanations, or get code snippets without modifying files directly.	• Understanding existing code • Debugging concepts • Generating small snippets to copy-paste
Plan	Architecture & Strategy Analyzes your request and codebase to build a step-by-step implementation plan (blueprint) without writing code immediately.	• Breaking down complex features • clarifying requirements before coding • Designing large refactors
Agent	Autonomous Execution An autonomous mode that can plan, edit, run terminal commands, and fix errors across multiple files to complete a task from start to finish.	• End-to-end feature implementation • Complex multi-file tasks • Tasks requiring iterative testing and fixing



Add to context

- VS Code automatically adds the active file as a context item in the chat input box
- E.g. Suppose you select main.py and view it in editor
- You can drag additional files to the chat areas and use them as the context.
- # mentions
 - # followed by the file name, folder name, or symbol name



Starting the app

```
PS C:\programs\notetaking-app\src> python main.py
* Serving Flask app 'main'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use
a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://158.132.10.133:5001
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
* Debugger is active!
* Debugger PIN: 296-050-124
```

```
python3 -m venv venv
source venv/bin/activate

pip install -r requirements.txt
python src/main.py
```

- Test the app using the exposed URL by codespaces

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS 1				
Port	Forwarded Address	Running Process	Visibility	Origin
5001	https://reimagined-co...	/home/codespace/.python/current/bin/python s...	Private	Auto Forwarded
Add Port				

- Test the app and observe the backend logs

a production WSGI server instead.

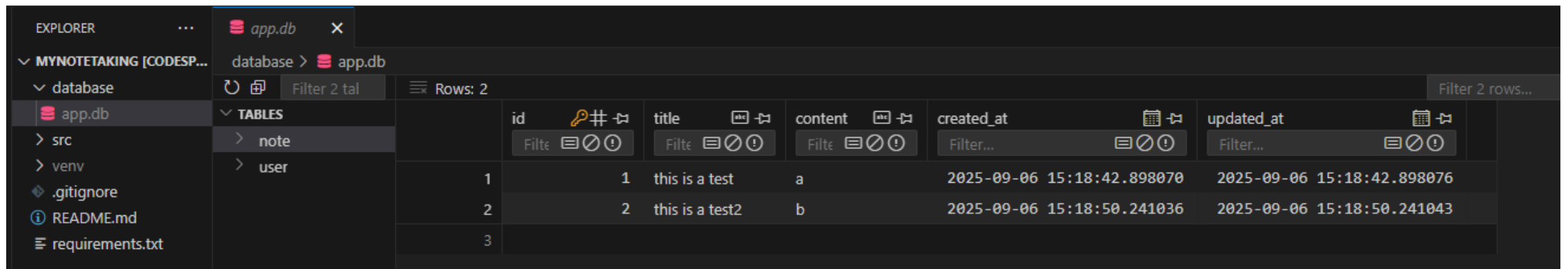
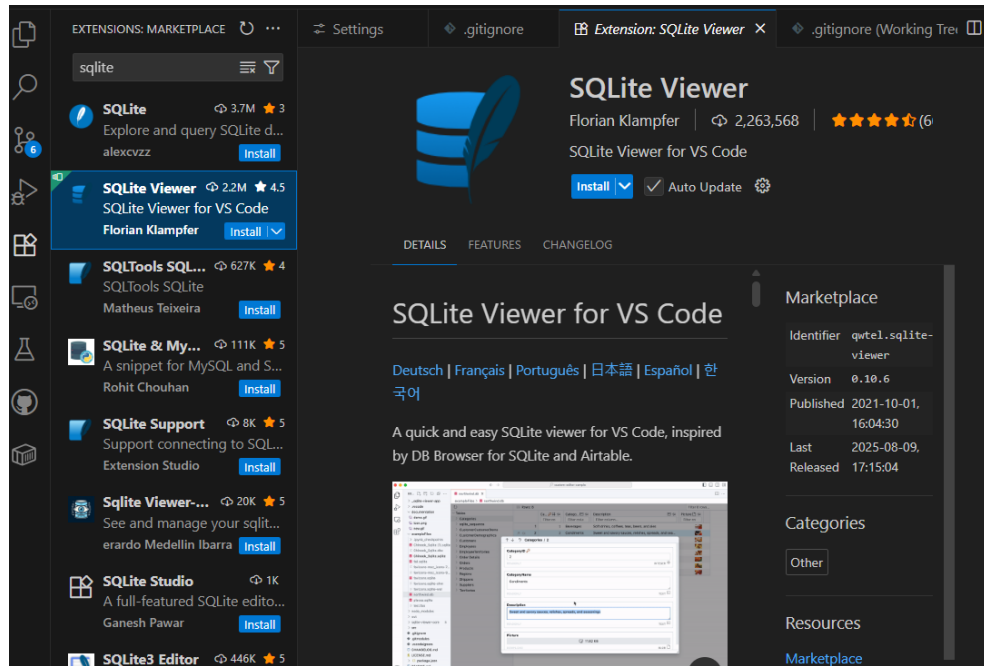
- * Running on all addresses (0.0.0.0)
- * Running on http://127.0.0.1:5001
- * Running on http://158.132.10.133:5001

Press CTRL+C to quit

- * Restarting with watchdog (windowsapi)
- * Debugger is active!
- * Debugger PIN: 296-050-124

```
127.0.0.1 - - [04/Sep/2025 12:35:00] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/Sep/2025 12:35:01] "GET /api/notes HTTP/1.1" 200 -
127.0.0.1 - - [04/Sep/2025 12:35:01] "GET /favicon.ico HTTP/1.1" 200 -
127.0.0.1 - - [04/Sep/2025 12:35:09] "POST /api/notes HTTP/1.1" 201 -
127.0.0.1 - - [04/Sep/2025 12:35:09] "PUT /api/notes/3 HTTP/1.1" 200 -
127.0.0.1 - - [04/Sep/2025 12:35:12] "GET / HTTP/1.1" 304 -
127.0.0.1 - - [04/Sep/2025 12:35:12] "GET /api/notes HTTP/1.1" 200 -
127.0.0.1 - - [04/Sep/2025 12:35:13] "GET /favicon.ico HTTP/1.1" 304 -
127.0.0.1 - - [04/Sep/2025 12:35:19] "POST /api/notes HTTP/1.1" 201 -
127.0.0.1 - - [04/Sep/2025 12:35:20] "PUT /api/notes/4 HTTP/1.1" 200 -
```

- Install “SQLite Viewer” VSCode plugin
- View the database



Test the app and write down your test cases and the bugs/issues that you have found

Test Cases	Bugs/Issues

Git

- In GitHub Codespaces, any changes made within the environment will be lost once the environment is deleted because it is temporary.
- To ensure your work is saved, you should first commit your changes to the local Git repository. After committing, push these changes to the remote repository on GitHub.
- This process ensures that all your work is stored safely and can be accessed from any future environment.

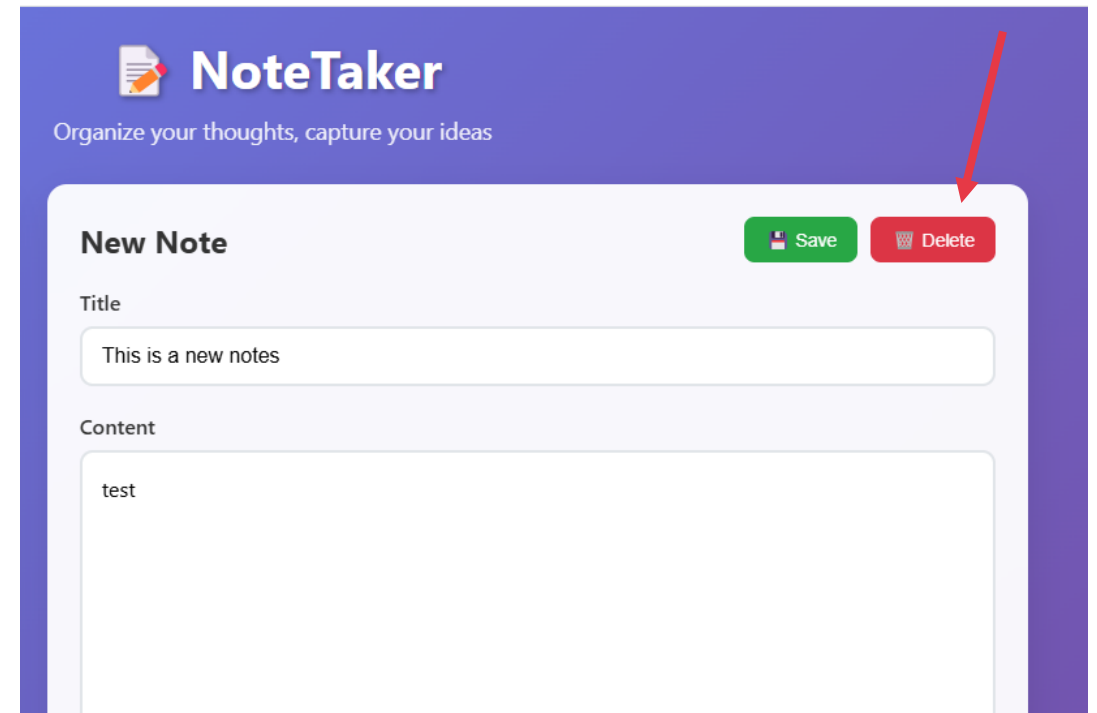
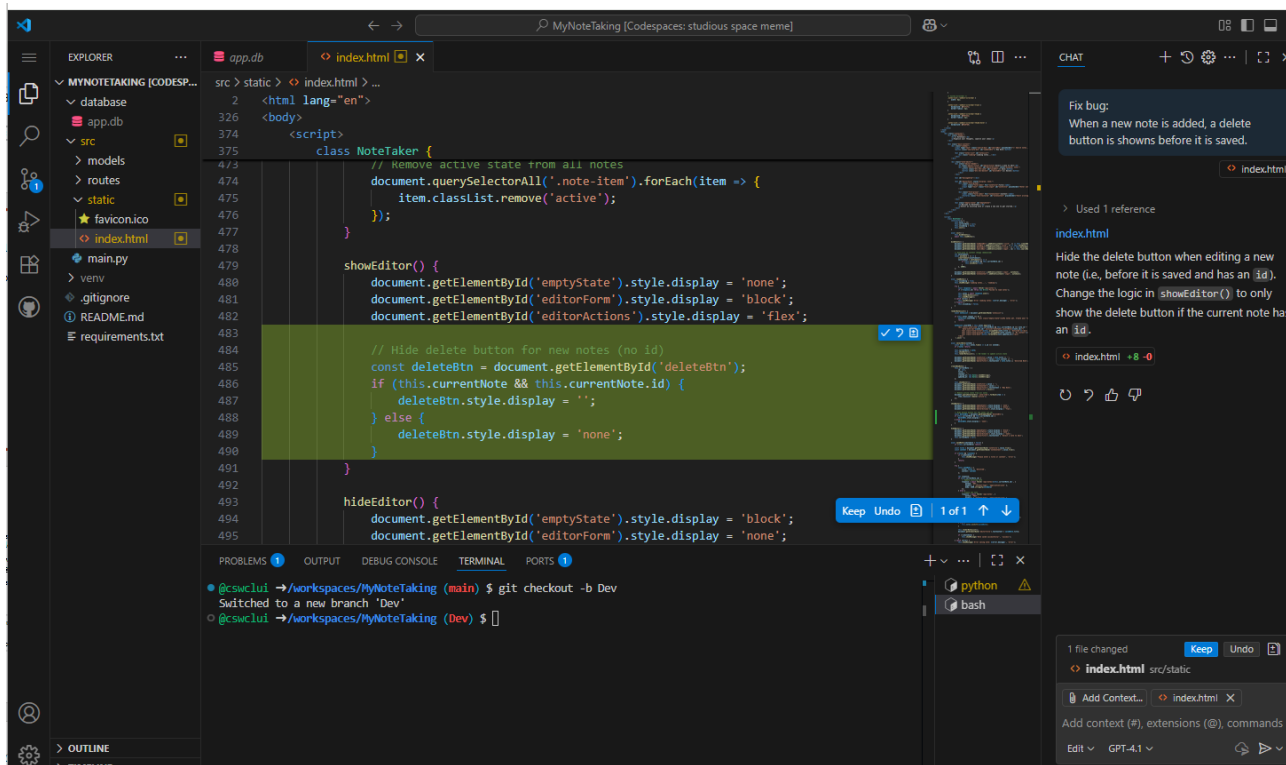
- **Git:** Version control system used to track changes in source code during software development
- **Commit:** A commit in Git is a snapshot of your changes in the repository.

Git Branch

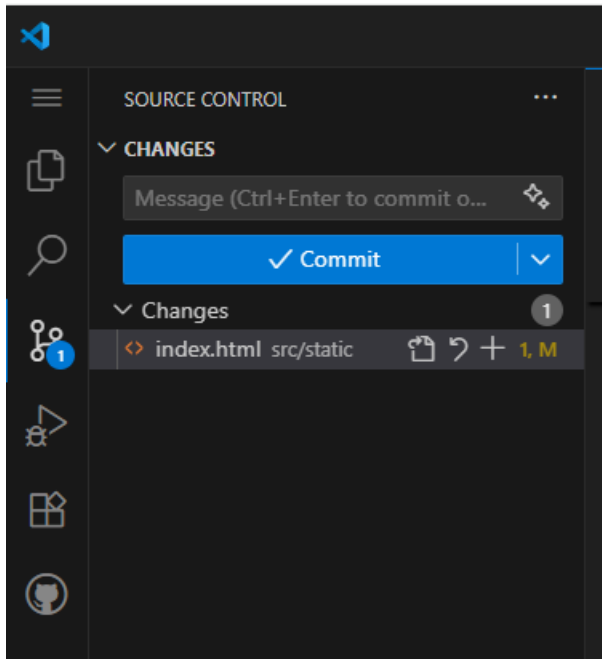
- **Branch:** A parallel version of the repository. It allows you to work on different features, fixes, or experiments independently from the main codebase.
- In Git, the use of branches allows you to diverge from the main line of development and continue to do work without affecting the main branch.
 - E.g., you may work on new features, bug fixes, or experiments without affecting the main codebase. Multiple developers can work on different tasks simultaneously without interfering with each other

Fix a bug

- Use the coding agent to mode to fix the bug that the delete button should not appear before the notes is saved.



- Create a new branch “Dev” and checkout to the branch
- Stage and commit the change with a descriptive commit message
- Push the new commit to the GitHub repository

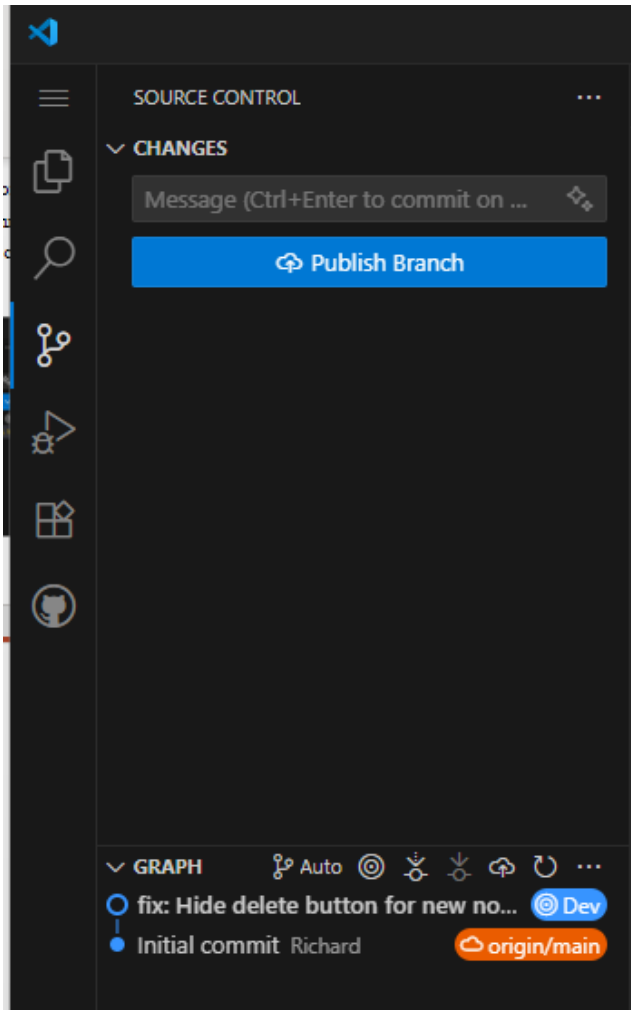


```
# Create and checkout to a new branch named "Dev"
git checkout -b Dev

# Stage your changes
git add .

# Commit with a descriptive message
git commit -m "fix: Hide delete button for new notes in the editor"

# Push the new branch and commit to GitHub
git push origin Dev
```



```
# Push the new branch and commit to GitHub  
git push origin Dev
```

- Now, generate a pull request
 - A pull request is a formal proposal to merge code changes from one branch into another, allowing for review, discussion, and approval before integration.
- Request Copilot to generate a summary

 Dev had recent pushes 53 seconds ago

Compare & pull request

**Add a title**

Fix: Newline rendering in notes list

Add a description

WritePreview



Add your description here...

Generate

Summary
Generate a summary of the changes in this pull request.

 Markdown is supported Paste, drop, or click to add files

Create pull request

R
S
E
A
N
L
N
P
N
N
D
U
a

- You may request a team member/copilot to review the code change

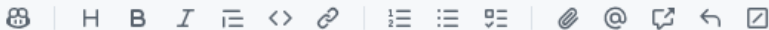


Add a title

Fix: Newline rendering in notes list

Add a description

WritePreview



This pull request makes a small but important improvement to the note preview rendering in the `src/static/index.html` file. The change ensures that line breaks in note content are displayed correctly in the note preview section.

* Note preview content now replaces newline characters with `
` tags, allowing multi-line notes to be displayed with proper formatting in the preview. `[[1]](diffhunk://#diff-6c71709b4d31c8f650dbe64ffc24d55cede1d64448e827c4b1292ffeb2a3acc3L439-R441)` `[[2]](diffhunk://#diff-6c71709b4d31c8f650dbe64ffc24d55cede1d64448e827c4b1292ffeb2a3acc3L592-R596)`

 Markdown is supported

 Paste, drop, or click to add files

Create pull request

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

Reviewers

Suggestions

 CopilotRequest

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Use [Closing keywords](#) in the description to automatically close issues

Helpful resources

[GitHub Community Guidelines](#)

16

- Review the suggestion of the Copilot coding agent.
- Merge the pull request

Copilot AI reviewed 2 minutes ago [View reviewed changes](#)

Copilot AI left a comment

Pull Request Overview

This PR fixes newline rendering in note preview displays by converting newline characters to HTML `<br>` tags. This ensures that multi-line notes display with proper line breaks in the preview section instead of appearing as single-line text.

- Modifies note preview rendering to replace `\n` characters with `<br>` tags
- Applied consistently to both note list rendering sections in the HTML file

Tip: Customize your code reviews with copilot-instructions.md. [Create the file](#) or [learn how to get started](#).

src/static/index.html

```

... @@ -436,7 +436,9 @@ <h3>Welcome to NoteTaker!</h3>
436 436 <div class="note-item" ${this.currentNote && this.currentNote.id === note.id ? 'active' : ''}
437 437   data-note-id="${note.id}" onclick="noteTaker.selectNote(${note.id})">
438 438   <div class="note-title">${this.escapeHtml(note.title || 'Untitled')}</div>
439 -   <div class="note-preview">${this.escapeHtml(note.content || 'No content')}</div>
439 +   <div class="note-preview">${
440 +     this.escapeHtml((note.content || 'No content')).replace(/\n/g, '<br>')}
  
```

Copilot AI 2 minutes ago

The newline replacement with `<br>` tags happens after HTML escaping, which means the `<br>` tags will be displayed as literal text instead of functioning as line breaks. Move the newline replacement before `escapeHtml()` or use a different approach to preserve line breaks while maintaining security.

Suggested change

```

440 -   this.escapeHtml((note.content || 'No content')).replace(/\n/g, '<br>')
440 +   this.escapeHtml(note.content || 'No content').replace(/\n/g, '<br>')
  
```

[Commit suggestion](#) [Add suggestion to batch](#)

No conflicts with base branch
Merging can be performed automatically.

[Merge pull request](#) You can also merge this with the command line. [View command line instructions.](#)

✓ **Create a merge commit**
All commits from this branch will be added to the base branch via a merge commit.

Squash and merge
The 3 commits from this branch will be combined into one commit in the base branch.

Rebase and merge
The 3 commits from this branch will be rebased and added to the base branch.

Commit message
Merge pull request #1 from PolyUTest/Dev

Extended description
Fix Newline rendering in notes list

This commit will be authored by 218137+cswdui@users.noreply.github.com.

[Confirm merge](#) [Cancel](#)

Task: Develop a translation feature for the notes

Prepare BBQ Chinese Translate Save Delete

Title

Prepare BBQ

Content

Items:

- Pork
- Chicken
- Vegetable



准备烧烤 Chinese Translate Save Delete

Title

准备烧烤

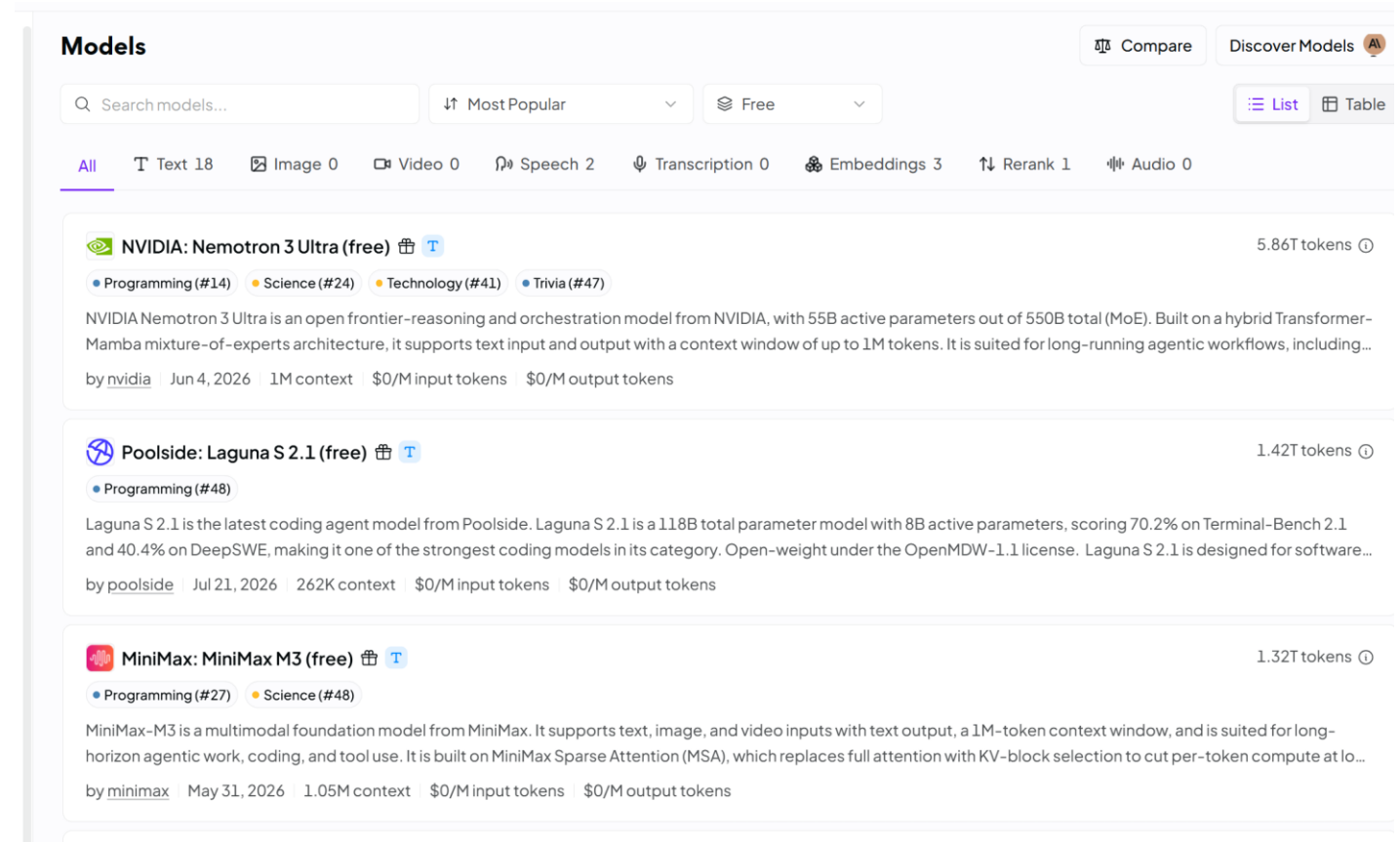
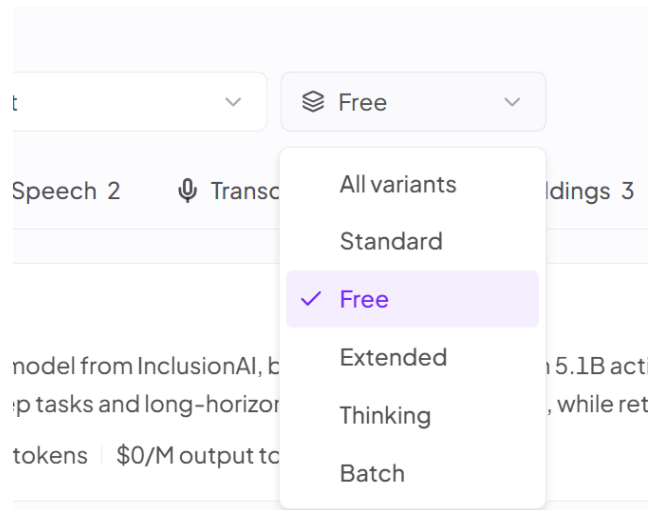
Content

物品：

- 猪肉
- 鸡肉
- 蔬菜

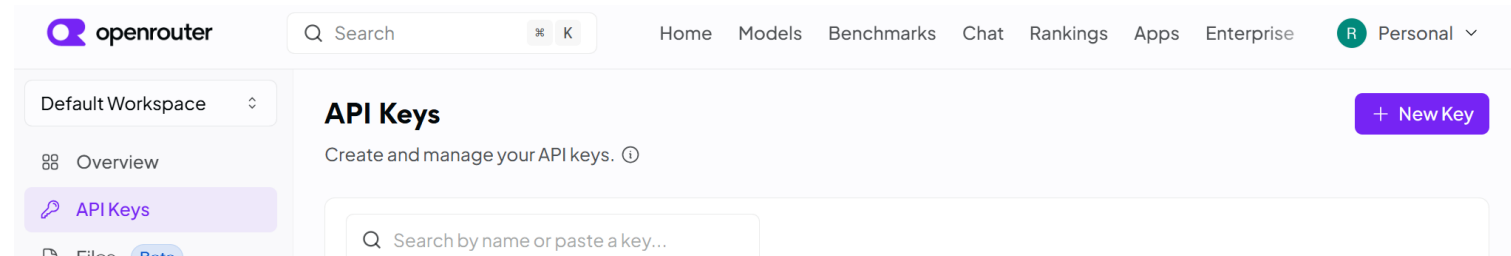
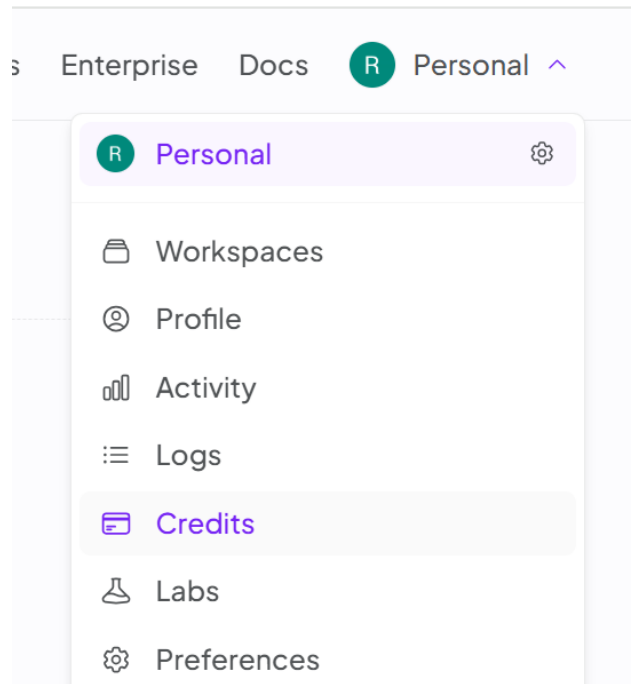
Openrouter LLM APIs

- Free models are available
- US \$1 Free quota
 - Budget can get to ~\$1



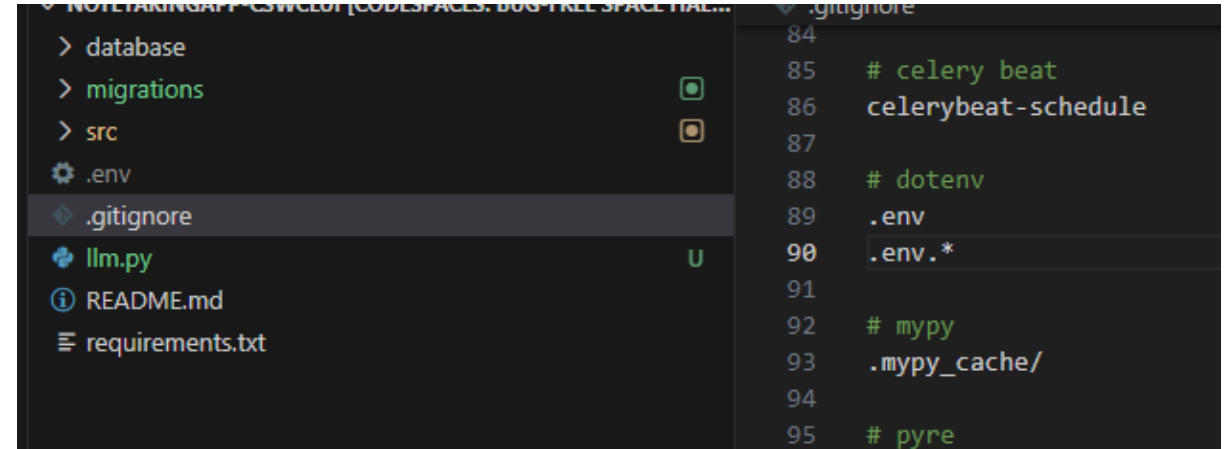
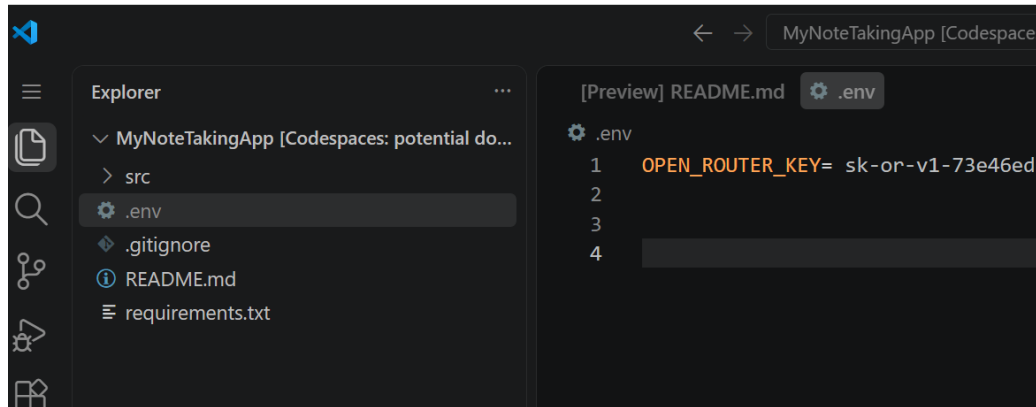
<https://openrouter.ai/models>

Get an API Key



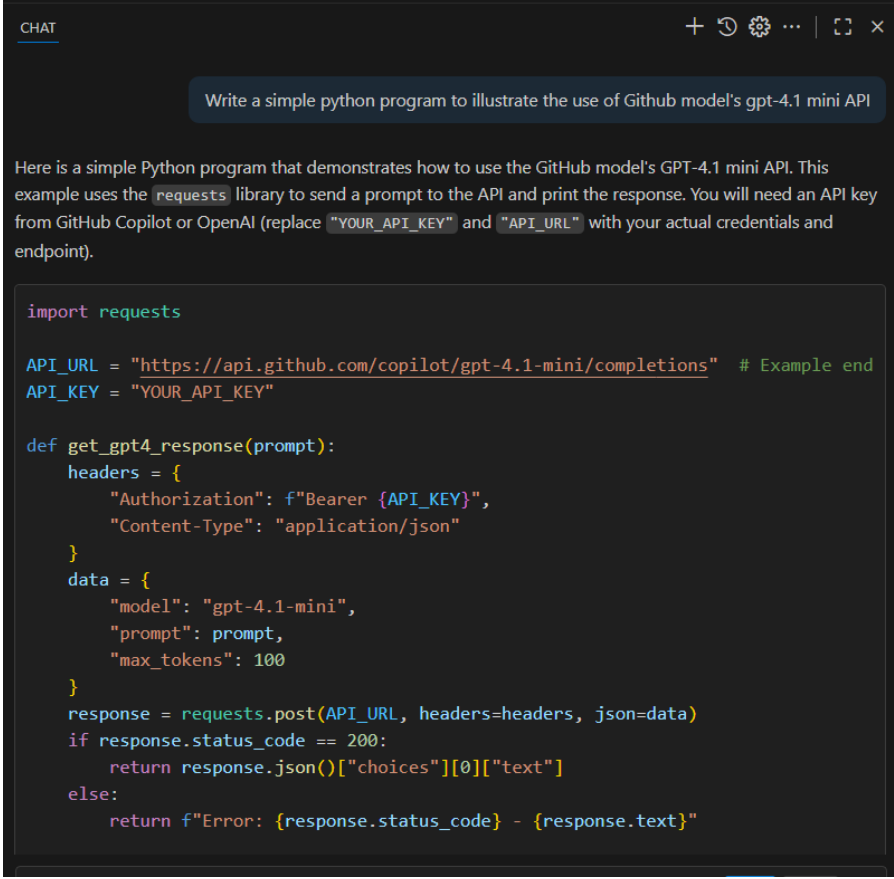
Storing API keys/credentials

- Store the GitHub Model Credential in .env file
- Make sure the **.env** is specified in **.gitignore** file and the **.env** is not tracked by git.



Limitations: Using LLM for coding

- Large language models (LLMs) are trained on large code corpora collected at a fixed point in time, LLMs often lack real-time updates on library evolution and API changes.
 - As a result, they may suggest code that use deprecated libraries or outdated APIs that may no longer function correctly with the latest versions of libraries or frameworks.
 - The generated code is more prone to hallucination, and might fail to compile or run, resulting in compatibility issues, bugs, or even security vulnerabilities.
- Solution:
 - Provide sample code for LLM as the context to write code



```
CHAT
+ 🔄 ⚙️ ... | 🗂️ ✕

Write a simple python program to illustrate the use of Github model's gpt-4.1 mini API

Here is a simple Python program that demonstrates how to use the GitHub model's GPT-4.1 mini API. This example uses the requests library to send a prompt to the API and print the response. You will need an API key from GitHub Copilot or OpenAI (replace "YOUR_API_KEY" and "API_URL" with your actual credentials and endpoint).

import requests

API_URL = "https://api.github.com/copilot/gpt-4.1-mini/completions" # Example end
API_KEY = "YOUR_API_KEY"

def get_gpt4_response(prompt):
    headers = {
        "Authorization": f"Bearer {API_KEY}",
        "Content-Type": "application/json"
    }
    data = {
        "model": "gpt-4.1-mini",
        "prompt": prompt,
        "max_tokens": 100
    }
    response = requests.post(API_URL, headers=headers, json=data)
    if response.status_code == 200:
        return response.json()["choices"][0]["text"]
    else:
        return f"Error: {response.status_code} - {response.text}"
```

Is the code correct?

Choose a LLM model (cheaper/free models)

Models					
Search models...					Top Weekly
All Text 22 Image 34 Video 29 Speech 2 Transcription 0 Embeddings 3 Rerank 7 Audio 2					
Model Name	Weekly Tokens	Input	Output	Context	
NVIDIA: Nemotron 3 Ultra (free)	3.75T	\$0	\$0	1,000,000	
Poolside: Laguna S 2.1 (free)	1.25T	\$0	\$0	262,144	
InclusionAI: Ling 3.0 Flash Fin (free)	1.06T	\$0	\$0	262,144	
NVIDIA: Nemotron 3.5 Lightning (free)	753B	\$0	\$0	1,000,000	
Dots Studio: Dots3-Note Preview (free)	701B	\$0	\$0	512,000	
Nex AGI: Nex-N2.5-Pro (free)	465B	\$0	\$0	262,144	
Thinking Machines: Inking (free)	418B	\$0	\$0	1,048,576	
NVIDIA: Nemotron 3 Super (free)	391B	\$0	\$0	262,144	
InclusionAI: Ling 3.0 Flash Sante (free)	374B	\$0	\$0	262,144	
InclusionAI: Ling 3.0 Flash VL (free)	322B	\$0	\$0	262,144	
Thinking Machines: Inking Small (free)	138B	\$0	\$0	1,048,576	
Nex AGI: Nex-N2.5-Mini (free)	136B	\$0	\$0	262,144	
Cohere: North Mini Code (free)	131B	\$0	\$0	256,000	
VoyageAI by MongoDB: rerank-2.5-lite	101B		\$0.02	32,000	
Poolside: Laguna XS 2.1 (free)	94.5B	\$0	\$0	262,144	
NVIDIA: Nemotron 3 Nano Omni (free)	38B	\$0	\$0	256,000	

- Study the sample code for Python (OpenAI)

Quick Start

Drop-in code to call this model with OpenRouter's OpenAI-compatible API.

1 Get your API key

Create an API key from your OpenRouter dashboard and set it as an environment variable:

+ Create API Key

```
export OPENROUTER_API_KEY=sk-or-v1- ...
```

Copy

2 Make your first request

Use `minimax/minimax-m3:free` with the OpenRouter API:

OpenRouter supports reasoning-enabled models that can show their step-by-step thinking process. Use the `reasoning` parameter in your request to enable reasoning, and access the `reasoning_details` array in the response to see the model's internal reasoning before the final answer. When continuing a conversation, preserve the complete `reasoning_details` when passing messages back to the model so it can continue reasoning from where it left off. [Learn more about reasoning tokens](#).

In the examples below, the [OpenRouter-specific headers](#) are optional. Setting them allows your app to appear on the OpenRouter leaderboards.

TypeScript SDK Python TypeScript (fetch) cURL **Python (OpenAI)** TypeScript (OpenAI)

```
from openai import OpenAI

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key="<OPENROUTER_API_KEY>",
)

# First API call with reasoning
response = client.chat.completions.create(
    model="minimax/minimax-m3:free",
    messages=[
        {
```

Copy

https://openrouter.ai/models?max_price=0&input_modalities=text&order=top-weekly

For cheaper model, you can select use:

<https://openrouter.ai/deepseek/deepseek-v4-flash-0731>

Learn about the LLM API

5

Edit Pins Watch 0 Fork 0

- Visit https://github.com/HKPolyUSE/LLM_To_AI_Agents
- Fork the repository into your own user account and start codespaces to launch the NextJS app in the terminal.

Run in terminal:

```
cd web
npm install
npm run dev
```

LLM_To_AI_Agents Public template

Edit Pins Watch 0 Fork 0 Star 0 Use this template

main 1 Branch 0 Tags

Go to file

Add file Code

Richard update e7ad99c · 3 minutes ago 2 Commits

web update 3 minutes ago

.gitignore Publish the LLM-to-agent lab: Next.js tutorial plus Python scri... 25 minutes ago

README.md update 3 minutes ago

agent.py Publish the LLM-to-agent lab: Next.js tutorial plus Python scri... 25 minutes ago

About

No description, website, or topics provided.

Readme

Activity

Custom properties

0 stars

0 watching

0 forks

Create a new repository

Open in a codespace

From LLM API to Coding Agent

Home

LLM API

Conversation history

Tool Use

Coding agent

API endpoint

https://openrouter.ai/api/v1/ch

Model

deepseek/deepseek-v4-flash-

API key

Missing

Paste API key

Leave a field blank to use the value from .env. A filled field overrides it.

LLM API

Let's start with a chatbot that replies in ordinary English. We send the user's question to a hosted language model and get text back. I will show you what the LLM API is, what we put in the request, and what a completion is.

1.1 The LLM API

Let's say you want a chatbot that answers in ordinary English. You do not write every reply yourself. You use a **large language model (LLM)**. It reads the text you send and writes text back.

A strong model is too large to run on your laptop. So you send the user's question to a hosted model, and the host sends a reply. That call is the **LLM API**.

An **API** is a contract. You send a request in a set format. You get a response. You do not need to know how the server is built.

On the web that contract is **HTTP**. A request has a **URL**, a **method** (this lab uses POST), **headers** (including an API key), and a **body**. The server returns a status code and a **response body**. In this lab both bodies are JSON.

This lab calls **OpenRouter**. OpenRouter is a gateway. You talk to one OpenAI-compatible endpoint and pay from one prepaid credit balance. Behind that, it can reach hundreds of models from many providers. You send one request. OpenRouter forwards it to the model and returns the reply.

Put the key in a `.env` file, not in your source code. This page reads that file from the lab folder, one level above `web/`. You can also type the endpoint, model, and key in the sidebar.

1.2 How the LLM API works

When you talk to a chat API, you usually send two kinds of text. The **system message** is written by you, the developer. It tells the model who it is and how it should behave. The **user message** is the question from the person using the chatbot.

Let's say the system message is "You are Alice, a helpful assistant. First say who you are, then answer." And the

Use the coding agent to write a program using the sample code

PROMPT:

read the API key from .env.

The program should define a function `llm_generate()` which accepts a user prompt read from the command line.

It should define a system prompt to act as a professional translator to translate the user input into the target language

Target language: {put your target language here}

Put your sample program below

{put the sample code from Openrouter here ...}

Run the program

- Add the library in requirements.txt
 - openai==1.106.1
 - dotenv==0.9.9
- Install the library in the virtual environment
 - pip install -r requirements.txt
- Run the app
 - python translator.py “How are you?”

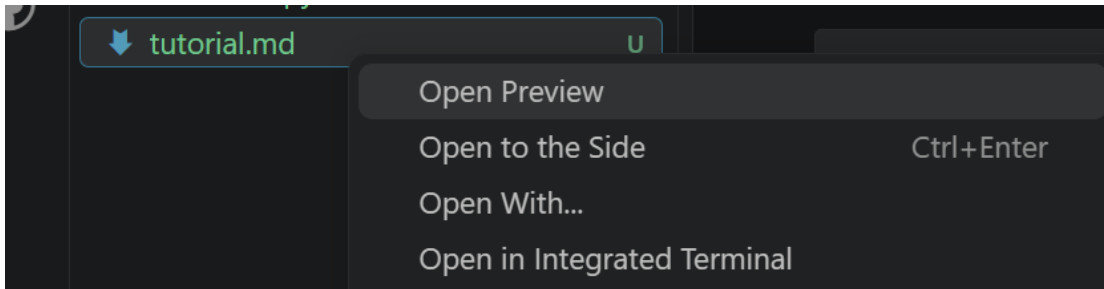
```
• @richardluipolyu → /workspaces/MyNoteTakingApp (main) $ python translator.py "How are you?"  
你好吗？
```

Use AI to create a tutorial to explain the program

- **Exercise:** Use the following prompt to ask AI to explain how translator.py works.

Write a tutorial to explain how this program works.
output in tutorial.md

- Preview the markdown file in VSCode (right click the file->Open Preview)



Step-by-step flow

a) Get the API key

```
api_key = _resolve_api_key()
```

b) Create the OpenAI client

```
client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=api_key,
)
```

This tells the Python OpenAI SDK to send requests to OpenRouter instead of OpenAI's default API.

c) Build a system prompt

```
system_prompt = (
    f"You are a professional translator. Translate the user's text into {TARGET_LANGUAGE}. "
    "Preserve the original meaning, tone, and intent. Return only the translated text."
)
```

This prompt tells the LLM:

- be a professional translator
- translate into the target language
- keep the meaning and tone
- return only the translation

d) Send the request to the model

```
response = client.chat.completions.create(
    model="minimax/minimax-m3:free",
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ],
    temperature=0.3,
)
```

≡ How translator.py talks to the LLM ≡ ☆

API key → OpenRouter client → system + user prompts → chat.completions.create

① Load the API key

.env

OPEN_ROUTER_KEY=sk-or-...

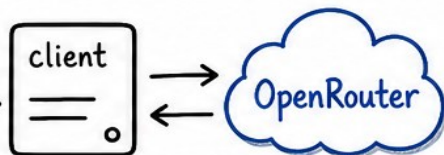
_resolve_api_key()

- Reads OPEN_ROUTER_KEY from the environment
- Returns that value

✗ If missing,
raise **ValueError**

② Create the OpenAI client

```
client = OpenAI(  
    base_url="https://openrouter.ai/api/v1",  
    api_key=api_key  
)
```



💡 OpenAI SDK, but
endpoint is OpenRouter,
not openai.com



api_key is sent as the
Authorization header
on every request.

③ Build the two prompts

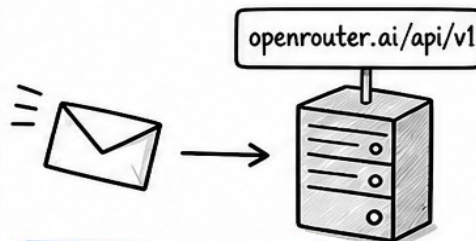
messages = [

① role: **system** —
"You are a professional
translator. Translate the
user's text into Chinese.
Preserve meaning, tone,
and intent. Return only
the translated text."

② role: **user** —
user_prompt
(the text typed on
the command line)

```
$ python translator.py  
"Hello world"
```

④ Send to the API endpoint



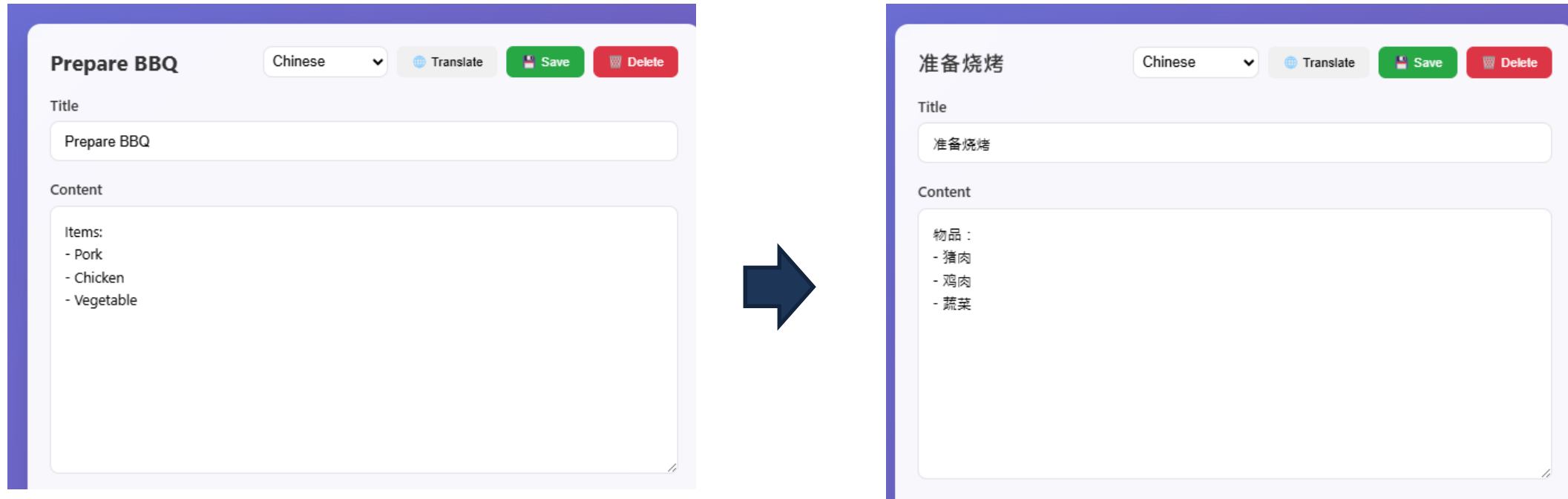
```
POST openrouter.ai/api/v1  
model = "minimax/minimax-m3:free"  
messages = messages  
temperature = 0.3
```

Reply:
response.choices[0].
message.content

⚠ If auth fails → **AuthenticationError**
→ tell user the key is invalid.

Develop a new feature to translate the notes using LLM APIs

1. Start a new session in the coding agent. Add **translator.py** as the context of the prompt for when writing the code
2. Prompt the coding agent (with translator.py as the context):
Create the translation feature in the note-taking app.
Store the prompt file in the folder prompts/translate_prompt.md, and the translation result should be output in JSON format



Try:

1. Use the plan mode to plan for the changes. Confirm/refine the plan before executing with the agent mode
2. Use AI to explain what code changes are made when creating the feature.
3. View the prompt file created.

Commit change in Dev branch

- Use the GitHub Copilot Agent mode to add the translation function to the App
- Stage the changes
- Commit with message **"feat: Translate notes"**

```
# Create and checkout to a new branch named "Dev"  
# If you already have the branch created before, use "git checkout Dev" to change to the  
branch
```

```
git checkout -b Dev
```

```
# Stage your changes and commit
```

```
git add .
```

```
git commit -m "feat: Translate notes"
```

```
# Push the new branch and commit to GitHub
```

```
git push origin Dev
```

Copilot instruction

- Defined in a markdown file named **.github/copilot-instructions.md**
 - Natural language guidance to help Github Copilot generate code and responses better tailored to a project's specific needs, team workflows, coding standards, and preferences.
 - E.g. You may define your coding style preference, how you want APIs called, naming patterns you want followed, etc
- The Copilot Agent will first read the file to understand the project contexts/rules before completing the actual task
- Purpose
 - Help align AI-generated code with human expectations, project standards, and team workflows.
 - Help the AI understand project-specific context, business logic, and domain knowledge

Example 1

This is a Next.js-based travel application with TypeScript that helps users search for trips, manage bookings, view travel guides, and track points. The application uses React components, server components, and client components as part of the Next.js App Router architecture. Please follow these guidelines when contributing:

Code Standards

Required Before Each Commit

- Run ``npm run lint`` to ensure code follows project standards
- Make sure all components follow Next.js App Router patterns
- Client components should be marked with `'use client'` when they use browser APIs or React hooks
- When adding new functionality, make sure you update the README
- Make sure that the repository structure documentation is correct and accurate in the Copilot Instructions file
- Ensure all tests pass by running ``npm run test`` in the terminal

TypeScript and React Patterns

- Use TypeScript interfaces/types for all props and data structures
- Follow React best practices (hooks, functional components)
- Use proper state management techniques
- Components should be modular and follow single-responsibility principle

Styling

- You must prioritize using Tailwind CSS classes as much as possible. If needed, you may define custom Tailwind Classes / Styles. Creating custom CSS should be the last approach.

Development Flow

- Install dependencies: ``npm install``
- Development server: ``npm run dev``
- Build: ``npm run build``
- Test: ``npm run test``
- Lint: ``npm run lint``

Development Flow

- Install dependencies: ``npm install``
- Development server: ``npm run dev``
- Build: ``npm run build``
- Test: ``npm run test``
- Lint: ``npm run lint``

Repository Structure

- ``app/``: Next.js App Router pages and layouts organized by route
- ``components/``: Reusable React components
 - ``components/ui/``: UI components (buttons, inputs, etc.)
 - ``components/__tests__/``: Component tests
- ``lib/``: Core logic and services
 - ``lib/data/``: Data models and mock data
 - ``lib/types/``: TypeScript type definitions
- ``public/``: Static assets
- ``tests/``: Test files and test utilities
- ``README.md``: Project documentation

Key Guidelines

1. Make sure to evaluate the components you're creating, and whether they need `'use client'`
2. Images should contain meaningful alt text unless they are purely for decoration. If they are for decoration only, a null (empty) alt text should be provided (`alt=""`) so that the images are ignored by the screen reader.
3. Follow Next.js best practices for data fetching, routing, and rendering
4. Use proper error handling and loading states
5. Optimize components and pages for performance

Example 2

Project Awareness & Context

- Always read `PLANNING.md` at the start of a new conversation to understand the project's architecture, goals, style, and constraints.
- Check `TASK.md` before starting a new task. If the task isn't listed, add it with a brief description and today's date.
- Use consistent naming conventions, file structure, and architecture patterns as described in `PLANNING.md`.
- Use `venv_linux` (the virtual environment) whenever executing Python commands, including for unit tests.

Code Structure & Modularity

- Never create a file longer than 500 lines of code. If a file approaches this limit, refactor by splitting it into modules or helper files.
- Organize code into clearly separated modules, grouped by feature or responsibility. For agents this looks like:
 - `agent.py` - Main agent definition and execution logic
 - `tools.py` - Tool functions used by the agent
 - `prompts.py` - System prompts
- Use clear, consistent imports (prefer relative imports within packages).
- Use clear, consistent imports (prefer relative imports within packages).
- Use `python_dotenv` and `load_env()` for environment variables.

Testing & Reliability

- Always create Pytest unit tests for new features (functions, classes, routes, etc).
- After updating any logic, check whether existing unit tests need to be updated. If so, do it.
- Tests should live in a `/tests` folder mirroring the main app structure.
 - Include at least:
 - 1 test for expected use
 - 1 edge case
 - 1 failure case

Task Completion

- Mark completed tasks in `TASK.md` immediately after finishing them.
- Add new sub-tasks or TODOs discovered during development to `TASK.md` under a "Discovered During Work" section.

Style & Conventions

- Use Python as the primary language.
- Follow PEP8, use type hints, and format with `black`.
- Use `pydantic` for data validation.
- Use `FastAPI` for APIs and `SQLAlchemy` or `SQLModel` for ORM if applicable.
- Write docstrings for every function using the Google style:

```
def example():
    """
    Brief summary.

    Args:
        param1 (type): Description.

    Returns:
        type: Description.
    """
```

Documentation & Explainability

- Update `README.md` when new features are added, dependencies change, or setup steps are modified.
- Comment non-obvious code and ensure everything is understandable to a mid-level developer.
- When writing complex logic, add an inline `# Reason:` comment explaining the why, not just the what.

AI Behavior Rules

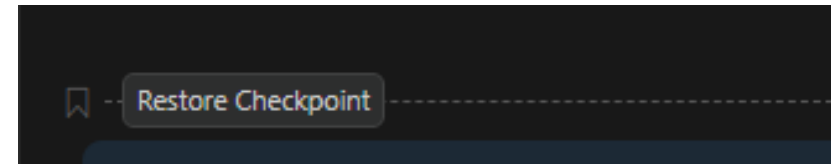
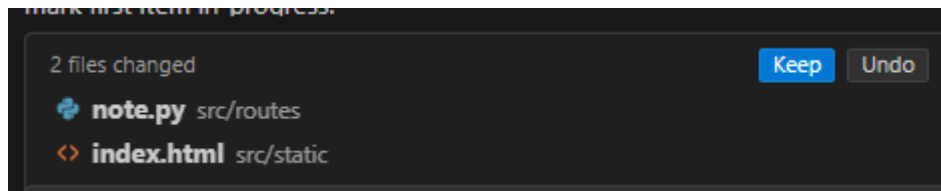
- Never assume missing context. Ask questions if uncertain.
- Never hallucinate libraries or functions – only use known, verified Python packages.
- Always confirm file paths and module names exist before referencing them in code or tests.
- Never delete or overwrite existing code unless explicitly instructed to or if part of a task from `TASK.md`.

Exercise

- Fix the remaining bugs of the app.
- Add new features to the app and commit the feature
 - E.g, change the background color of the app
- Try undoing the commit you have made
- Remark:
 - After the AI makes the code changes, you should test to see if the code changes is working correctly
 - Commit each change in the “Dev” branch
 - Push the commits and update the remote repository (GitHub)

Undoing the changes generated by AI

- “Undo” the AI changes
- “Restore Checkpoint” in the GitHub Copilot Chat
- Using “git reset” Command
 - Soft, Mixed vs. Hard reset



Exercise: Deploying your app to Vercel

- Let's deploy your notetaking app to <https://vercel.com>!
- However, our existing app makes use of SQLite for storing data (file-based database)
 - Vercel's serverless deployment model does not support the persistent, writable filesystem that SQLite depends on.
 - The filesystem available to serverless functions in Vercel is ephemeral (wiped on every request)
 - The backend functions are run using serverless functions, and files written to the function's local disk should be treated as temporary.
 - The app cannot reliably write to or read from a local database file
- Complete the following task with coding agents.
 - Refactor the app to make it stateless by storing the data in an external database
 - E.g., Use a cloud database (e.g. Neon, Postgres DB in Supabase/insforge)
 - You can install the agent skill in your repository for the corresponding platform (e.g. Neon, Supabase) to facilitate the agent coding to better write the integration code.
 - Test that the app can successfully store the data in the database
 - Refactor the app structure such that the translation can be run successfully in the Vercel cloud platform.
 - The LLM API key should be retrieved from the environment variable (configured in the Vercel platform)
 - Allow the user to attach images/documents to the notes (store in the object store in Neon/Supabase/Insforge)
 - Others:
 - Add unique features to your notetaking app to make it more useful and innovative.
 - Improve the UI/UX of the app

Resources

- <https://code.claude.com/docs/en/context-window>
- <https://academy.claude.com/tutorials/parametric-memory-and-context>
- <https://code.claude.com/docs/en/prompt-caching>
- Building AI Agents in Pure Python - Beginner Course
 - <https://www.youtube.com/watch?v=c9AnqCeyxbI>